

LAKSHMI NARAIN COLLEGE OF TECHNOLOGY

Department of Computer Science & Engineering

ASSIGNMENT REPORT

Submitted By

VARUN JUNEJA

College	LNCT
Enrollment Number	0103CS231449
Superset ID	7809995
Email	varunjuneja1905@gmail.com

Academic Session 2025 – 2026

LABORATORY PRACTICAL RECORD: SPRING FRAMEWORK & SPRING BOOT

Exercise 1: Configuring a Basic Spring Application

Category Spring Core

Aim To set up a basic Maven project and configure the Spring Application Context using XML to manage domain service and repository beans.

Concept Inversion of Control (IoC) delegates the responsibility of object instantiation and lifecycle management from application code to the Spring IoC container via metadata configuration.

Scenario A library management web application requires a modular backend architected using the Spring Framework to centralize object creation and manage foundational domain operations.

Steps Performed 1. Created a standard Maven project directory structure named `LibraryManagement`. 2. Configured the `pom.xml` file with `spring-context` dependencies. 3. Created domain packages for service and repository layers. 4. Implemented `BookRepository` and `BookService` Java classes. 5. Created `applicationContext.xml` inside `src/main/resources` defining class beans. 6. Developed a standalone main class to load `ClassPathXmlApplicationContext`. 7. Retrieved instantiated beans from the container and verified successful execution.

Project / Files Created * `pom.xml`

- `src/main/resources/applicationContext.xml`
- `src/main/java/com/library/repository/BookRepository.java`
- `src/main/java/com/library/service/BookService.java`
- `src/main/java/com/library/LibraryManagementApplication.java`

Source Code

File: `pom.xml`

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.library</groupId>
  <artifactId>LibraryManagement</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
    <spring.version>5.3.30</spring.version>
  </properties>
```

```
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>${spring.version}</version>
  </dependency>
</dependencies>
</project>
```

File: `src/main/resources/applicationContext.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-
beans.xsd">
  <bean id="bookRepository" class="com.library.repository.BookRepository"/>
  <bean id="bookService" class="com.library.service.BookService"/>
</beans>
```

File: `src/main/java/com/library/repository/BookRepository.java`

```
package com.library.repository;

public class BookRepository {
    public String fetchAllBooks() {
        return "Book Repository: Fetched list of available library books.";
    }
}
```

File: `src/main/java/com/library/service/BookService.java`

```
package com.library.service;

public class BookService {
    public void executeService() {
        System.out.println("Book Service: Executing foundational library
behaviors.");
    }
}
```

File: `src/main/java/com/library/LibraryManagementApplication.java`

```
package com.library;

import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context = new
        ClassPathXmlApplicationContext("applicationContext.xml");
        BookService service = (BookService) context.getBean("bookService");
        service.executeService();
    }
}
```

Compilation and Execution (Screenshot) The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 1: Compilation and execution output

[INSERT SCREENSHOT HERE]

Expected Console Output

```
May 12, 2026 10:15:00 AM
org.springframework.context.support.ClassPathXmlApplicationContext prepareRefresh
INFO: Refreshing
org.springframework.context.support.ClassPathXmlApplicationContext
Book Service: Executing foundational library behaviors.
```

Output / Result The basic Spring framework architecture was initialized successfully using Maven and XML configuration. The IoC container instantiated the service and repository beans autonomously, verifying correct context bootstrapping.

Exercise 2: Implementing Dependency Injection

Category Spring Core (IoC & DI)

Aim To implement XML-based setter dependency injection to wire the `BookRepository` bean into the `BookService` bean.

Concept Dependency Injection (DI) allows the Spring container to inject dependent object instances into target beans at runtime, eliminating hardcoded couplings.

Scenario The backend service layer requires database access methods defined in the repository layer to fetch records without tightly coupling class constructors.

Steps Performed 1. Modified `BookService` to include a private `BookRepository` reference. 2. Generated a standard setter method inside `BookService` for container injection. 3. Updated `applicationContext.xml` with `<property>` tags to establish bean linkage. 4. Invoked repository methods through the service layer bean. 5. Compiled the application to verify clean dependency resolution. 6. Executed the main class and validated console results.

Project / Files Created * `src/main/resources/applicationContext.xml`

- `src/main/java/com/library/service/BookService.java`
- `src/main/java/com/library/LibraryManagementApplication.java`

Source Code

File: `src/main/resources/applicationContext.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-
beans.xsd">
    <bean id="bookRepository" class="com.library.repository.BookRepository"/>
    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>
</beans>
```

File: `src/main/java/com/library/service/BookService.java`

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {
    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void manageBooks() {
        System.out.println("BookService: Delegating data retrieval to Repository
layer.");
        System.out.println(bookRepository.fetchAllBooks());
    }
}
```

```
}
```

File: `src/main/java/com/library/LibraryManagementApplication.java`

```
package com.library;

import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml");
        BookService service = (BookService) context.getBean("bookService");
        service.manageBooks();
    }
}
```

Compilation and Execution (Screenshot) The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 2: Compilation and execution output

[INSERT SCREENSHOT HERE]

Expected Console Output

```
BookService: Delegating data retrieval to Repository layer.
Book Repository: Fetched list of available library books.
```

Output / Result Setter dependency injection was successfully configured via XML metadata. The Spring IoC container automatically injected the repository instance into the service bean, enabling loose coupling.

Exercise 3: Implementing Logging with Spring AOP

Category Spring AOP

Aim To implement Aspect-Oriented Programming (AOP) using AspectJ annotations to intercept service executions and log runtime metrics.

Concept AOP modularizes cross-cutting concerns like logging and performance benchmarking dynamically via runtime proxies without altering core business logic.

Scenario To monitor system latency, the library backend requires an automated logging mechanism that calculates execution times across domain service methods.

Steps Performed 1. Added `spring-aop` and `aspectjweaver` dependencies into `pom.xml`. 2. Created `com.library.aspect` package and `LoggingAspect` class. 3. Annotated `LoggingAspect` with `@Aspect` and defined `@Around` advice. 4. Updated `applicationContext.xml` with `<aop:aspectj-autoproxy/>` configuration. 5. Registered `LoggingAspect` as a standard Spring bean. 6. Executed target service methods to trigger runtime advice interception.

Project / Files Created * `pom.xml`

- `src/main/resources/applicationContext.xml`
- `src/main/java/com/library/aspect/LoggingAspect.java`
- `src/main/java/com/library/LibraryManagementApplication.java`

Source Code

File: `pom.xml`

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.library</groupId>
  <artifactId>LibraryManagement</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.30</version>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-aop</artifactId>
      <version>5.3.30</version>
    </dependency>
    <dependency>
      <groupId>org.aspectj</groupId>
      <artifactId>aspectjweaver</artifactId>
      <version>1.9.7</version>
    </dependency>
  </dependencies>
</project>
```

File: src/main/resources/applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-
                           beans.xsd
                           http://www.springframework.org/schema/aop
                           http://www.springframework.org/schema/aop/spring-
                           aop.xsd">
    <aop:aspectj-autoproxy/>
    <bean id="bookRepository" class="com.library.repository.BookRepository"/>
    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>
    <bean id="loggingAspect" class="com.library.aspect.LoggingAspect"/>
</beans>
```

File: src/main/java/com/library/aspect/LoggingAspect.java

```
package com.library.aspect;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;

@Aspect
public class LoggingAspect {
    @Around("execution(* com.library.service.*.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable
    {
        long start = System.currentTimeMillis();
        Object proceed = joinPoint.proceed();
        long duration = System.currentTimeMillis() - start;
        System.out.println("[AOP LOG] Method " +
            joinPoint.getSignature().getName() + " executed in " + duration + " ms.");
        return proceed;
    }
}
```

File: src/main/java/com/library/LibraryManagementApplication.java

```
package com.library;
```

```
import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml");
        BookService service = (BookService) context.getBean("bookService");
        service.manageBooks();
    }
}
```

Compilation and Execution (Screenshot) The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 3: Compilation and execution output

[INSERT SCREENSHOT HERE]

Expected Console Output

```
BookService: Delegating data retrieval to Repository layer.
Book Repository: Fetched list of available library books.
[AOP LOG] Method manageBooks executed in 12 ms.
```

Output / Result Spring AOP successfully intercepted core business operations using AspectJ dynamic proxy weaving. Method performance metrics were captured non-invasively, separating cross-cutting logging from infrastructure code.

Exercise 4: Creating and Configuring a Maven Project

Category Build Tools & Framework Setup

Aim To construct a complete Maven project definition equipped with enterprise Spring dependencies and Java compiler plugins.

Concept Maven utilizes a Project Object Model (`pom.xml`) to standardize build lifecycles, enforce Java version compliance, and resolve transitive repository dependencies.

Scenario A new enterprise library application requires a stable build configuration capable of compiling Java 1.8 code and packaging MVC core binaries.

Steps Performed 1. Bootstrapped a standardized Maven project named `LibraryManagement`. 2. Declared project coordinates (`groupId`, `artifactId`, `version`). 3. Added `spring-context`, `spring-aop`, and `spring-webmvc` dependencies. 4. Configured `maven-compiler-plugin` targeting Java 1.8 specifications. 5. Validated POM XML syntax and structure. 6. Executed a clean compilation cycle via build tool commands.

Project / Files Created * pom.xml**Source Code**

File: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.library</groupId>
  <artifactId>LibraryManagement</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <name>Library Management Enterprise Build</name>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <java.version>1.8</java.version>
    <spring.version>5.3.30</spring.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>${spring.version}</version>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-aop</artifactId>
      <version>${spring.version}</version>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>${spring.version}</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.1</version>
        <configuration>
          <source>1.8</source>
          <target>1.8</target>
        </configuration>
      </plugin>
    </plugins>
```

```
</build>
</project>
```

Compilation and Execution (Screenshot) The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 4: Compilation and execution output

[INSERT SCREENSHOT HERE]

Expected Console Output

```
[INFO] Scanning for projects...
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @
LibraryManagement ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 4 source files to /workspace/LibraryManagement/target/classes
[INFO] BUILD SUCCESS
```

Output / Result The Maven build configuration was verified successfully. All Spring core framework dependencies were resolved, and the compiler plugin executed cleanly under Java 1.8 constraints.

Exercise 5: Configuring the Spring IoC Container

Category Spring Core

Aim To establish a centralized XML configuration container defining application beans and wiring domain layers through setter methods.

Concept The Spring IoC container reads wiring metadata to instantiate, assemble, and manage application components across their entire execution lifecycle.

Scenario The backend library platform demands centralized container wiring to eliminate manual object instantiation across layered business modules.

Steps Performed 1. Created `applicationContext.xml` within `src/main/resources`. 2. Defined explicit `<bean>` definitions for service and repository classes. 3. Configured `<property>` dependency injection tags inside the XML container. 4. Ensured robust setter mutator exposure in `BookService`. 5. Bootstrapped the container via `ClassPathXmlApplicationContext`. 6. Tested domain logic flow from service down to repository layers.

Project / Files Created * `src/main/resources/applicationContext.xml`

- `src/main/java/com/library/service/BookService.java`
- `src/main/java/com/library/LibraryManagementApplication.java`

Source Code

File: src/main/resources/applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-
beans.xsd">
    <bean id="bookRepository" class="com.library.repository.BookRepository"/>
    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>
</beans>
```

File: src/main/java/com/library/service/BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {
    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void verifyContainerWiring() {
        System.out.println("IoC Container: Initializing central configuration
test.");
        System.out.println(bookRepository.fetchAllBooks());
    }
}
```

File: src/main/java/com/library/LibraryManagementApplication.java

```
package com.library;

import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml");
```

```
        BookService service = (BookService) context.getBean("bookService");
        service.verifyContainerWiring();
    }
}
```

Compilation and Execution (Screenshot) The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 5: Compilation and execution output

[INSERT SCREENSHOT HERE]

Expected Console Output

```
IoC Container: Initializing central configuration test.
Book Repository: Fetched list of available library books.
```

Output / Result Centralized Spring IoC container management was successfully established. The XML setup initialized domain layers autonomously, proving dependable architectural decoupling.

Exercise 6: Configuring Beans with Annotations

Category Spring Core (Annotations)

Aim To simplify container metadata configuration using `@Service`, `@Repository`, and automated component scanning.

Concept Stereotype annotations tag classes as Spring-managed components, enabling the IoC container to discover and register beans automatically via classpath scanning.

Scenario To accelerate development, the library platform requires replacing verbose XML declarations with stereotype annotations across domain classes.

Steps Performed 1. Configured `<context:component-scan>` inside `applicationContext.xml`. 2. Annotated `BookRepository` with `@Repository` stereotype. 3. Annotated `BookService` with `@Service` and `@Autowired` fields. 4. Removed explicit `<bean>` tags from XML metadata files. 5. Loaded the context and fetched dynamically scanned beans. 6. Verified annotation-driven wiring execution via console logging.

Project / Files Created * `src/main/resources/applicationContext.xml`

- `src/main/java/com/library/repository/BookRepository.java`
- `src/main/java/com/library/service/BookService.java`
- `src/main/java/com/library/LibraryManagementApplication.java`

Source Code

File: `src/main/resources/applicationContext.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-
                           beans.xsd
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-
                           context.xsd">
    <context:component-scan base-package="com.library"/>
</beans>
```

File: src/main/java/com/library/repository/BookRepository.java

```
package com.library.repository;

import org.springframework.stereotype.Repository;

@Repository
public class BookRepository {
    public String fetchAllBooks() {
        return "Annotation Scan: BookRepository successfully detected and wired.";
    }
}
```

File: src/main/java/com/library/service/BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class BookService {
    @Autowired
    private BookRepository bookRepository;

    public void verifyAnnotationScan() {
        System.out.println("Annotation Scan: BookService operational.");
        System.out.println(bookRepository.fetchAllBooks());
    }
}
```

File: `src/main/java/com/library/LibraryManagementApplication.java`

```
package com.library;

import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context = new
        ClassPathXmlApplicationContext("applicationContext.xml");
        BookService service = context.getBean(BookService.class);
        service.verifyAnnotationScan();
    }
}
```

Compilation and Execution (Screenshot) The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 6: Compilation and execution output

[INSERT SCREENSHOT HERE]

Expected Console Output

```
Annotation Scan: BookService operational.
Annotation Scan: BookRepository successfully detected and wired.
```

Output / Result Annotation-based configuration successfully replaced verbose XML definitions. Component scanning detected stereotype tags automatically, maintaining complete structural integrity.

Exercise 7: Implementing Constructor and Setter Injection

Category Spring Core (Advanced DI)

Aim To implement and verify both constructor-based and setter-based dependency injection within a single Spring application context.

Concept Constructor injection enforces mandatory dependency fulfillment at instantiation, whereas setter injection facilitates optional runtime re-configuration.

Scenario The business domain requires mandatory repository injection via constructors, alongside optional configuration parameters applied via setter methods.

Steps Performed 1. Defined a parameterized constructor inside `BookService` accepting `BookRepository`. 2. Created a secondary setter method inside `BookService` for string metadata configuration. 3. Updated `applicationContext.xml` using `<constructor-arg>` tags. 4. Added `<property>` tags to configure setter fields concurrently. 5. Bootstrapped the application context using the driver class. 6. Verified dual-injection behavior through console output verification.

Project / Files Created * `src/main/resources/applicationContext.xml`

- `src/main/java/com/library/service/BookService.java`
- `src/main/java/com/library/LibraryManagementApplication.java`

Source Code

File: `src/main/resources/applicationContext.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-
                           beans.xsd">
    <bean id="bookRepository" class="com.library.repository.BookRepository"/>
    <bean id="bookService" class="com.library.service.BookService">
        <constructor-arg ref="bookRepository"/>
        <property name="libraryName" value="Central Engineering Library"/>
    </bean>
</beans>
```

File: `src/main/java/com/library/service/BookService.java`

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {
    private final BookRepository bookRepository;
    private String libraryName;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void setLibraryName(String libraryName) {
        this.libraryName = libraryName;
    }

    public void executeDualInjection() {
        System.out.println("Active Facility: " + libraryName);
        System.out.println("Constructor DI: Wired Repository successfully.");
    }
}
```

```
        System.out.println(bookRepository.fetchAllBooks());  
    }  
}
```

File: `src/main/java/com/library/LibraryManagementApplication.java`

```
package com.library;  
  
import com.library.service.BookService;  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
public class LibraryManagementApplication {  
    public static void main(String[] args) {  
        ApplicationContext context = new  
ClassPathXmlApplicationContext("applicationContext.xml");  
        BookService service = (BookService) context.getBean("bookService");  
        service.executeDualInjection();  
    }  
}
```

Compilation and Execution (Screenshot) The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 7: Compilation and execution output

[INSERT SCREENSHOT HERE]

Expected Console Output

```
Active Facility: Central Engineering Library  
Constructor DI: Wired Repository successfully.  
Book Repository: Fetched list of available library books.
```

Output / Result Both constructor and setter injection patterns were configured successfully. The container enforced immutable mandatory dependencies while allowing flexible optional mutator configurations.

Exercise 8: Implementing Basic AOP with Spring

Category Spring AOP

Aim To construct basic AspectJ advice methods separating cross-cutting logging execution before and after business logic processing.

Concept AOP separates non-functional cross-cutting concerns from core domain logic by defining pointcuts that trigger advice blocks during method execution lifecycles.

Scenario To ensure audit compliance, the library management platform requires automated transaction logging triggered before and after domain service calls.

Steps Performed 1. Created `LoggingAspect` class within `com.library.aspect` package. 2. Implemented `@Before` advice to log pre-execution parameters. 3. Implemented `@After` advice to log completion confirmations. 4. Configured `<aop:aspectj-autoproxy/>` within XML metadata files. 5. Registered aspect components inside the Spring IoC container. 6. Executed business invocations to verify clean intercept weaving.

Project / Files Created * `src/main/resources/applicationContext.xml`

- `src/main/java/com/library/aspect/LoggingAspect.java`
- `src/main/java/com/library/service/BookService.java`
- `src/main/java/com/library/LibraryManagementApplication.java`

Source Code

File: `src/main/resources/applicationContext.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-
                           beans.xsd
                           http://www.springframework.org/schema/aop
                           http://www.springframework.org/schema/aop/spring-
                           aop.xsd">
    <aop:aspectj-autoproxy/>
    <bean id="bookRepository" class="com.library.repository.BookRepository"/>
    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>
    <bean id="loggingAspect" class="com.library.aspect.LoggingAspect"/>
</beans>
```

File: `src/main/java/com/library/aspect/LoggingAspect.java`

```
package com.library.aspect;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
```

```
@Aspect
public class LoggingAspect {
    @Before("execution(* com.library.service.BookService.issueBook(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println("[AOP BEFORE] Initiating audit log for: " +
            joinPoint.getSignature().getName());
    }

    @After("execution(* com.library.service.BookService.issueBook(..))")
    public void logAfter(JoinPoint joinPoint) {
        System.out.println("[AOP AFTER] Transaction finalized for: " +
            joinPoint.getSignature().getName());
    }
}
```

File: `src/main/java/com/library/service/BookService.java`

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {
    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void issueBook() {
        System.out.println("Core Domain Logic: Executing book issuance
protocol...");
    }
}
```

File: `src/main/java/com/library/LibraryManagementApplication.java`

```
package com.library;

import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context = new
        ClassPathXmlApplicationContext("applicationContext.xml");
        BookService service = (BookService) context.getBean("bookService");
    }
}
```

```
        service.issueBook();  
    }  
}
```

Compilation and Execution (Screenshot) The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 8: Compilation and execution output

[INSERT SCREENSHOT HERE]

Expected Console Output

```
[AOP BEFORE] Initiating audit log for: issueBook  
Core Domain Logic: Executing book issuance protocol...  
[AOP AFTER] Transaction finalized for: issueBook
```

Output / Result Basic AOP pointcuts and advice methods were successfully integrated. The application cleanly separated audit logging concerns from foundational business operations.

Exercise 9: Creating a Spring Boot Application

Category Spring Boot & REST API

Aim To develop a Spring Boot REST API backed by Spring Data JPA and an H2 in-memory database to manage library book records.

Concept Spring Boot auto-configures underlying framework complexity, embedded servers, and data sources, enabling rapid RESTful microservice delivery.

Scenario The library platform demands a modern REST architectural layer capable of handling CRUD operations seamlessly over HTTP endpoints.

Steps Performed 1. Bootstrapped a Spring Boot project utilizing Spring Initializr. 2. Included `spring-boot-starter-web`, `spring-boot-starter-data-jpa`, and `h2` dependencies. 3. Configured H2 database properties inside `application.properties`. 4. Created domain entity `Book` mapped with standard JPA annotations. 5. Created repository interface extending `JpaRepository`. 6. Implemented `BookController` defining standard REST HTTP mappings. 7. Executed the application and verified operations via simulated HTTP payloads.

Project / Files Created * `pom.xml`

- `src/main/resources/application.properties`
- `src/main/java/com/library/model/Book.java`
- `src/main/java/com/library/repository/BookRepository.java`
- `src/main/java/com/library/controller/BookController.java`

- `src/main/java/com/library/LibraryManagementApplication.java`

Source Code

File: `pom.xml`

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.7.15</version>
  </parent>
  <groupId>com.library</groupId>
  <artifactId>LibraryManagement</artifactId>
  <version>1.0.0</version>
  <properties>
    <java.version>1.8</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
  </dependencies>
</project>
```

File: `src/main/resources/application.properties`

```
spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
server.port=8080
```

File: `src/main/java/com/library/model/Book.java`

```
package com.library.model;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String author;

    public Book() {}

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
}
```

File: `src/main/java/com/library/repository/BookRepository.java`

```
package com.library.repository;

import com.library.model.Book;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import org.springframework.stereotype.Repository;

@Repository
public interface BookRepository extends JpaRepository<Book, Long> {
}
```

File: `src/main/java/com/library/controller/BookController.java`

```
package com.library.controller;

import com.library.model.Book;
import com.library.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {
    @Autowired
    private BookRepository bookRepository;

    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return bookRepository.save(book);
    }

    @GetMapping
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }
}
```

File: `src/main/java/com/library/LibraryManagementApplication.java`

```
package com.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryManagementApplication {
    public static void main(String[] args) {
        SpringApplication.run(LibraryManagementApplication.class, args);
    }
}
```

```
}
```

Compilation and Execution (Screenshot) The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 9: Compilation and execution output

[INSERT SCREENSHOT HERE]

Expected Console Output

```

      .
  /\ /  _ _ _ _ _ ( _ ) _ _ _ _ _ \ \ \ \
( ( ) \ _ _ | ' _ | ' _ | ' _ \ _ _ | \ \ \ \
 \ \ / _ _ ) | | _ | | | | | | | ( _ | ) ) ) )
  ' | _ _ | . _ | | | | | _ \ , | / / / /
=====|_|=====|_|/=//_/_/_/
:: Spring Boot ::                (v2.7.15)

INFO 12452 --- [main] c.l.LibraryManagementApplication : Started
LibraryManagementApplication in 3.412 seconds

```

Endpoint Verification & REST Interaction

- **Endpoint URL:** `http://localhost:8080/api/books`
- **HTTP Method:** `POST`
- **Sample Request Headers:** `Content-Type: application/json`
- **Sample Request Body:**

```
{
  "title": "Clean Code",
  "author": "Robert C. Martin"
}
```

- **Sample Response Body:**

```
{
  "id": 1,
  "title": "Clean Code",
  "author": "Robert C. Martin"
}
```

- **Endpoint URL:** `http://localhost:8080/api/books`
- **HTTP Method:** `GET`
- **Sample Response Body:**

```
[  
  {  
    "id": 1,  
    "title": "Clean Code",  
    "author": "Robert C. Martin"  
  }  
]
```

Output / Result The Spring Boot REST API microservice was successfully deployed. Auto-configuration linked embedded Tomcat servers, Spring MVC controllers, and in-memory H2 JPA persistence layers seamlessly.